

Chapter 6: Cache

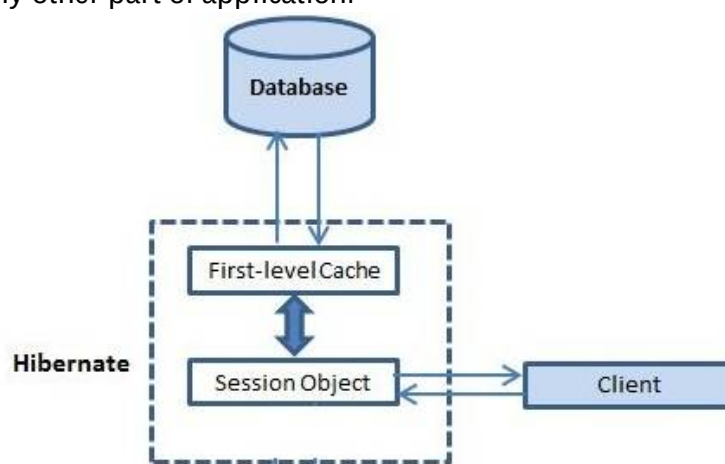
- Introduction
- First Level Cache
- Second Level Cache

Introduction

- Caching is a facility provided by ORM frameworks which help users to get fast running web application, while help framework itself to reduce number of queries made to database in a single transaction.
- Hibernate also provide this caching functionality, in two layers.
 - o First level cache
 - o Second level cache

First Level Cache

- First level cache in hibernate is enabled by default and you do not need to do anything to get this functionality working. In fact, you cannot disable it even forcefully.
- It is easy to understand the first level cache if we understand the fact that it is associated with Session object. As we know session object is created on demand from session factory and it is lost, once the session is closed. Similarly, first level cache associated with session object is available only till session object is live. It is available to session object only and is not accessible to any other session object in any other part of application.



Important facts

- First level cache is associated with “session” object and other session objects in application cannot see it.
- The scope of cache objects is of session. Once session is closed, cached objects are gone forever.
- First level cache is enabled by default and you cannot disable it.
- When we query an entity first time, it is retrieved from database and stored in first level cache associated with hibernate session.
- If we query same object again with same session object, it will be loaded from cache and no sql query will be executed.
- The loaded entity can be removed from session using **evict()** method. The next loading of this entity will again make a database call if it has been removed using **evict()** method.
- The whole session cache can be removed using **clear()** method. It will remove all the entities stored in cache.

Removing cache objects from first level cache example

Though we cannot disable the first level cache in hibernate, but we can certainly remove some of objects from it when needed. This is done using two methods:

- `evict()`
- `clear()`

Here `evict()` is used to remove a particular object from cache associated with session, and `clear()` method is used to remove all cached objects associated with session. So they are essentially like remove one and remove all.

Example: test11 (First Level Cache)

Pre-Requisite: Create a database by the **hib** in **MySQL 8**

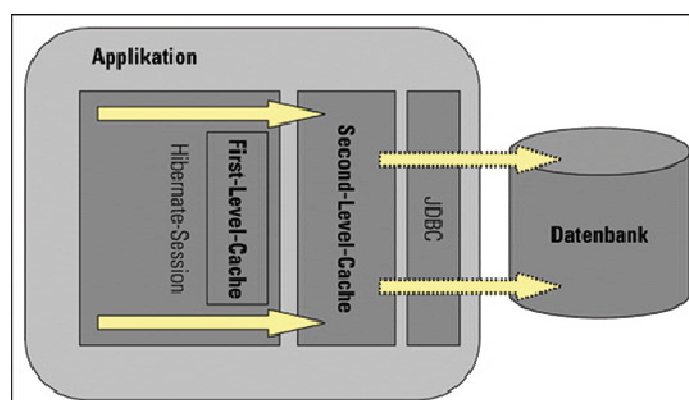
1. test11\pom.xml
2. src\main\java\hibernate.cfg.xml
3. com.vedisoft.hibernate.test11.utils
 - **HibernateUtil.java**
4. com.vedisoft.hibernate.test11.pojos
 - **Employee.java** @Entity
5. com.vedisoft.hibernate.test11.daos
 - **IEmployeeDao.java**
 - **EmployeeDao.java**
6. com.vedisoft.hibernate.test11
 - **App.java**

Second Level Cache

- Second level cache is created in session factory scope and is available to be used in all sessions which are created using that particular session factory.
- It also means that once session factory is closed, all cache associated with it die and cache manager also closed down.

How second level cache works

- If an instance is already present in the first-level cache, it is returned from there
- If an instance is not found in the first-level cache, and the corresponding instance state is cached in the second-level cache, then the data is fetched from there and an instance is assembled and returned
- Otherwise, the necessary data are loaded from the database and an instance is assembled and returned



Region Factory

- Hibernate second-level caching is designed to be unaware of the actual cache provider used.
- Hibernate only needs to be provided with an implementation of the `org.hibernate.cache.spi.RegionFactory` interface which encapsulates all details specific to actual cache providers. Basically, it acts as a bridge between Hibernate and cache providers.
- Some common cache providers are:
 - o EH Cache
 - o OS Cache
 - o Swarm Cache
 - o JBoss Cache

We will use Ehcache as a cache provider, which is a mature and widely used cache.

Cache Concurrency Strategy

1. **read-only**: caching will work for read only operation.
2. **nonstrict-read-write**: caching will work for read and write but one at a time.
3. **read-write**: caching will work for read and write, can be used simultaneously.
4. **transactional**: caching will work for transaction.

Implementation (Ehcache)

1. pom.xml
2. hibernate.cfg.xml

```
<property name="hibernate.cache.use_second_level_cache">true</property>
<property name="hibernate.cache.region.factory_class">
org.hibernate.cache.ehcache.EhCacheRegionFactory</property>
<property name="net.sf.ehcache.configurationResourceName">/ehcache.xml</property>
```

/ ehcache.xml

```
<ehcache xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:noNamespaceSchemaLocation="ehcache.xsd">

  <cache name="com.vedisoft.hibernate.test12.pojos.Employee"
maxElementsInMemory="10000" eternal="false" timeToIdleSeconds="300"
timeToLiveSeconds="600" overflowToDisk="true" />

</ehcache>
```

3. Entity and Query

Entity

```
import org.hibernate.annotations.Cache;
import org.hibernate.annotations.CacheConcurrencyStrategy;

@Entity
@Cache(usage=CacheConcurrencyStrategy.READ_WRITE)
public class Employee {}
```

Query

```
CriteriaBuilder cb = session.getCriteriaBuilder();
CriteriaQuery<Employee> criteria = cb.createQuery(Employee.class);
Root<Employee> root = criteria.from(Employee.class);
criteria.select(root);
Query<Employee> query = session.createQuery(criteria).setCacheable(true);
List<Employee> empList = query.getResultList();
```

Example: test12 (Second Level Cache)

Pre-Requisite: Create a database by the **hib** in **MySQL 8**

1. test12**pom.xml**
2. src\main\java**hibernate.cfg.xml**
3. com.vedisoft.hibernate.test12.utils
 - **HibernateUtil.java**
4. com.vedisoft.hibernate.test12.pojos
 - **Employee.java** @Entity
5. com.vedisoft.hibernate.test12.daos
 - **IEmployeeDao.java**
 - **EmployeeDao.java**
6. com.vedisoft.hibernate.test12
 - **App.java**